

Dưới đây là câu trả lời cho 5 câu hỏi của bạn:

1. **C allows _____ of code and data.**

Trả lời: C allows **sharing** of code and data.

Giải thích: Ngôn ngữ C hỗ trợ việc chia sẻ mã (code) và dữ liệu thông qua các cơ chế như hàm, thư viện, hoặc con trỏ, cho phép tái sử dụng và truy cập dữ liệu hiệu quả.

2. **A _____ is a diagrammatic representation that illustrates the sequence of operations to be performed to arrive at a solution.**

Trả lời: A **flowchart** is a diagrammatic representation that illustrates the sequence of operations to be performed to arrive at a solution.

Giải thích: Lưu đồ (flowchart) là một biểu diễn đồ họa thể hiện trình tự các bước hoặc thao tác cần thực hiện để giải quyết một vấn đề.

3. **Flowcharts help us review and debug programs easily. (True / False)**

Trả lời: **True**.

Giải thích: Lưu đồ giúp lập trình viên hình dung luồng chương trình, xác định lỗi logic, và đánh giá chương trình một cách trực quan, từ đó hỗ trợ việc xem xét (review) và gỡ lỗi (debug) dễ dàng hơn.

4. **A flowchart can have any number of start and stop points. (True / False)**

Trả lời: **False**.

Giải thích: Theo quy tắc chuẩn, một lưu đồ nên có **một điểm bắt đầu (start)** và **một điểm kết thúc (stop)** để đảm bảo tính rõ ràng và logic. Việc có nhiều hơn một điểm bắt đầu hoặc kết thúc có thể gây nhầm lẫn và không được khuyến khích trong thiết kế lưu đồ.

5. **A _____ is basically the execution of a sequence of statements until a particular condition is True or False.**

Trả lời: A **loop** is basically the execution of a sequence of statements until a particular condition is True or False.

Giải thích: Vòng lặp (loop) là cấu trúc trong lập trình cho phép thực thi lặp lại một chuỗi câu lệnh cho đến khi một điều kiện cụ thể trở thành True hoặc False, ví dụ như vòng lặp `for`, `while`, hoặc `do-while` trong C.

Dưới đây là câu trả lời cho 7 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **C is case sensitive. (TRUE / False)**

Trả lời: **TRUE**.

Giải thích: Ngôn ngữ lập trình C là **phân biệt hoa thường** (case-sensitive). Điều này có nghĩa là các biến hoặc từ khóa như `int`, `INT`, hoặc `Int` được coi là khác nhau. Ví dụ: `variable` và `Variable` là hai biến khác nhau trong C.

2. **The number 10 is a _____.**

Trả lời: The number 10 is a **constant**.

****Giải thích**:** Trong C, số 10 là một ****hằng số**** (constant), cụ thể là một hằng số nguyên (integer literal). Nó không thể thay đổi giá trị trong chương trình. Ví dụ: `int x = 10;` – ở đây, `10` là hằng số.

3. ****The first character of the identifier can be a number. (TRUE / False)****

****Trả lời**:** ****FALSE****.

****Giải thích**:** Theo quy tắc đặt tên trong C, ****định danh**** (identifier) như tên biến, tên hàm phải bắt đầu bằng chữ cái (A-Z hoặc a-z) hoặc dấu gạch dưới (`_`). Số (0-9) không thể đứng ở vị trí đầu tiên. Ví dụ: `int 123abc;` là không hợp lệ, nhưng `int abc123;` thì hợp lệ.

4. ****Using the type _____ saves memory as it takes only half the space as a _____****

****Trả lời**:** Using the type ****short**** saves memory as it takes only half the space as a ****long****.

****Giải thích**:**

- Trong C, kiểu dữ liệu `short` thường chiếm 2 byte (16 bit), trong khi `long` thường chiếm 4 byte (32 bit) trên nhiều hệ thống (tuy nhiên, kích thước có thể thay đổi tùy thuộc vào kiến trúc).

- Vì vậy, `short` tiết kiệm bộ nhớ hơn, chiếm khoảng một nửa không gian so với `long`.

5. ****The _____ data type is used to indicate the C compiler that no value is being returned.****

****Trả lời**:** The ****void**** data type is used to indicate the C compiler that no value is being returned.

****Giải thích**:** Kiểu dữ liệu `void` được sử dụng trong C để chỉ ra rằng một hàm không trả về giá trị nào. Ví dụ: `void myFunction() { ... }` – hàm này không trả về giá trị (khác với các hàm trả về kiểu `int`, `float`, v.v.).

6. ****_____ and _____ are the two classes of arithmetic operators.****

****Trả lời**:** ****Unary**** and ****Binary**** are the two classes of arithmetic operators.

****Đáp án đúng**:** B. Unary and Binary.

****Giải thích**:**

- Toán tử số học trong C được chia thành hai loại:

- ****Toán tử đơn ngôi (Unary)****: Chỉ cần một toán hạng, ví dụ: `++`, `--`, `-` (phủ định). Ví dụ: `x = -5;` hoặc `x++`.

- ****Toán tử nhị phân (Binary)****: Cần hai toán hạng, ví dụ: `+`, `-`, `*`, `/`, `%`. Ví dụ: `x = 5 + 3;`.

- Các lựa chọn khác như "Bitwise" hoặc "Logical" không phải là phân loại chính của toán tử số học.

7. ****The unary arithmetic operators are _____ and _____****

****Trả lời**:** The unary arithmetic operators are ****+**** and ****-****.

****Đáp án đúng**:** A. + and -.

****Giải thích**:**

- Toán tử đơn ngôi số học trong C bao gồm:

- `+` (dấu dương, ví dụ: `+5`).

- `-` (dấu âm/phủ định, ví dụ: `-5`).

- Ngoài ra, `++` và `--` cũng là toán tử đơn ngôi, nhưng trong ngữ cảnh "arithmetic operators" cơ bản, `+` và `-` thường được đề cập đến. Các lựa chọn khác như `%` hoặc `^` không phải toán tử đơn ngôi.

Tóm tắt câu trả lời:

1. C is case sensitive: **TRUE**.
2. The number 10 is a **constant**.
3. The first character of the identifier can be a number: **FALSE**.
4. Using the type **short** saves memory as it takes only half the space as a **long**.
5. The **void** data type is used to indicate the C compiler that no value is being returned.
6. **Unary** and **Binary** are the two classes of arithmetic operators (B).
7. The unary arithmetic operators are **++** and **--** (A).

1. **Operators** are the tools that manipulate data.

Trả lời: **Operators** are the tools that manipulate data.

Đáp án đúng: A. Operators.

Giải thích: Toán tử (operators) trong C là các công cụ dùng để thao tác dữ liệu, chẳng hạn như thực hiện các phép tính toán (+, -, *, /), so sánh (==, !=), hoặc thao tác bit (&, |). Ví dụ: `x = 5 + 3;` – toán tử `+` thao tác trên dữ liệu `5` và `3`.

2. An **expression** consists of a combination of operators and operands.

Trả lời: An **expression** consists of a combination of operators and operands.

Đáp án đúng: A. Expression.

Giải thích: Một biểu thức (expression) trong C là sự kết hợp của toán tử (operators) và toán hạng (operands) để tạo ra một giá trị. Ví dụ: `5 + 3` là một biểu thức, trong đó `5` và `3` là toán hạng, và `+` là toán tử.

3. **Precedence** establishes the hierarchy of one set of operators over another when an arithmetic expression has to be evaluated.

Trả lời: **Precedence** establishes the hierarchy of one set of operators over another when an arithmetic expression has to be evaluated.

Đáp án đúng: B. Precedence.

Giải thích: Độ ưu tiên (precedence) xác định thứ tự thực hiện của các toán tử trong một biểu thức số học. Ví dụ: Trong biểu thức `3 + 4 \* 2`, toán tử `\*` có độ ưu tiên cao hơn `+`, nên `4 \* 2` được tính trước, kết quả là `3 + 8 = 11`.

4. **Mixed Mode Expression** is one in which the operands of an operator belong to different data types.

Trả lời: **Mixed Mode Expression** is one in which the operands of an operator belong to different data types.

Đáp án đúng: B. Mixed Mode Expression.

Giải thích: Biểu thức hỗn hợp (mixed mode expression) là biểu thức mà các toán hạng của toán tử có kiểu dữ liệu khác nhau. Ví dụ: `int x = 5; float y = 3.14; float z = x + y;` – đây là biểu thức hỗn hợp vì `x` là `int` và `y` là `float`. C sẽ tự động chuyển đổi kiểu (type conversion) để thực hiện phép tính.

5. ****An expression can be forced to be of a certain type by using a _____.****

****Trả lời**:** An expression can be forced to be of a certain type by using a ****cast****.

****Đáp án đúng**:** A. Cast.

****Giải thích**:** Ép kiểu (cast) được sử dụng để ép một biểu thức thành một kiểu dữ liệu cụ thể. Trong C, điều này được thực hiện bằng cách đặt kiểu dữ liệu trong dấu ngoặc trước biểu thức. Ví dụ: `float x = (float)5 / 2;` – ở đây, `5` được ép kiểu thành `float`, nên phép chia trả về `2.5` thay vì `2`.

6. ****_____ are used to combine or negate expressions containing relational operators.****

****Trả lời**:** ****Logical Operators**** are used to combine or negate expressions containing relational operators.

****Đáp án đúng**:** A. Logical Operators.

****Giải thích**:** Toán tử logic (logical operators) như `&&` (AND), `||` (OR), và `!` (NOT) được sử dụng để kết hợp hoặc phủ định các biểu thức chứa toán tử quan hệ (như `>`, `<`, `==`). Ví dụ: `if (x > 5 && y < 10)` – toán tử `&&` kết hợp hai biểu thức quan hệ `x > 5` và `y < 10`.

Tóm tắt câu trả lời:

- **Operators**** are the tools that manipulate data (A).
- An ****expression**** consists of a combination of operators and operands (A).
- **Precedence**** establishes the hierarchy of one set of operators over another (B).
- **Mixed Mode Expression**** is one in which the operands of an operator belong to different data types (B).
- An expression can be forced to be of a certain type by using a ****cast**** (A).
- **Logical Operators**** are used to combine or negate expressions containing relational operators (A).

7. ****Bitwise logical operators are _____, _____, and _____.****

****Trả lời**:** Bitwise logical operators are ****&****, ****|****, and ****^****.

****Đáp án đúng**:** C. `&`, `|`, and `^`.

****Giải thích**:**

- Toán tử logic bitwise (bitwise logical operators) trong C thao tác trên các bit của toán hạng. Các toán tử này bao gồm:

- `&` (bitwise AND): Thực hiện phép AND trên từng cặp bit. Ví dụ: `5 & 3` (`0101 & 0011 = 0001`) trả về 1.

- `|` (bitwise OR): Thực hiện phép OR trên từng cặp bit. Ví dụ: `5 | 3` (`0101 | 0011 = 0111`) trả về 7.

- `^` (bitwise XOR): Thực hiện phép XOR trên từng cặp bit. Ví dụ: `5 ^ 3` (`0101 ^ 0011 = 0110`) trả về 6.

- Các lựa chọn khác không chính xác vì:

-similarityA, B, và D không liệt kê đầy đủ các toán tử bitwise.

8. ****The precedence of operators can be overridden by enclosing the required part of the expression in _____.****

****Trả lời**:** The precedence of operators can be overridden by enclosing the required part of the expression in ****parentheses****.

****Đáp án đúng**:** C. Parentheses.

****Giải thích**:**

- Độ ưu tiên (precedence) của toán tử xác định thứ tự thực hiện các phép toán trong biểu thức. Tuy nhiên, bạn có thể thay đổi thứ tự này bằng cách sử dụng dấu ngoặc tròn `()` để bao quanh phần biểu thức cần ưu tiên thực hiện trước.

- Ví dụ: Trong biểu thức `3 + 4 * 2`, `*` có độ ưu tiên cao hơn `+`, nên kết quả là `3 + (4 * 2) = 11`. Nhưng nếu bạn viết `(3 + 4) * 2`, thì `3 + 4` được tính trước, kết quả là `7 * 2 = 14`.

- Dấu ngoặc tròn giúp kiểm soát thứ tự thực hiện rõ ràng, không phụ thuộc vào độ ưu tiên của toán tử.

Tóm tắt câu trả lời:

7. Bitwise logical operators are ****&**, **|**, and **^**** (C).

8. The precedence of operators can be overridden by enclosing the required part of the expression in ****parentheses**** (C).

1. ****The formatted I/O functions are _____ and _____.****

****Trả lời**:** The formatted I/O functions are ****printf()**** and ****scanf()****.

****Đáp án đúng**:** A. printf() and scanf().

****Giải thích**:**

- Trong C, các hàm I/O định dạng (formatted I/O functions) là các hàm cho phép đọc/ghi dữ liệu theo định dạng cụ thể.

- `printf()` dùng để xuất dữ liệu có định dạng ra màn hình (stdout), ví dụ: `printf("So: %d", 10);`.

- `scanf()` dùng để đọc dữ liệu có định dạng từ bàn phím (stdin), ví dụ: `scanf("%d", &x);`.

- Các hàm như `getchar()` và `putchar()` là I/O không định dạng (unformatted), nên không thuộc nhóm này.

2. ****scanf() uses _____ to variables rather than variable names.****

****Trả lời**:** scanf() uses ****pointers**** to variables rather than variable names.

****Đáp án đúng**:** B. Pointers.

****Giải thích**:**

- Hàm `scanf()` yêu cầu truyền địa chỉ của biến (thay vì tên biến) để lưu dữ liệu vào biến đó. Điều này được thực hiện bằng cách sử dụng con trỏ (pointers) thông qua toán tử `&`.

- Ví dụ: `int x; scanf("%d", &x);` – `&x` là địa chỉ của biến `x`, cho phép `scanf()` ghi giá trị vào biến.

3. ****_____ specify the form by which the values of the variables are to be input/printed.****

****Trả lời**:** ****Format specifiers**** specify the form by which the values of the variables are to be input/printed.

****Đáp án đúng**:** B. Format specifiers.

****Giải thích**:**

- Bộ định dạng (format specifiers) là các chuỗi đặc biệt dùng trong `printf()` và `scanf()` để chỉ định kiểu dữ liệu và cách hiển thị/đọc dữ liệu.

- Ví dụ: `%d` cho số nguyên, `%f` cho số thực, `%s` cho chuỗi, `%c` cho ký tự.

- Ví dụ: `printf("So: %d", 10);` – `%d` quy định giá trị `10` được in dưới dạng số nguyên.

4. **_____** is used by the `printf()` function to identify conversion specifications.

Trả lời: `%%` is used by the `printf()` function to identify conversion specifications.

Đáp án đúng: A. %.

Giải thích:

- Trong `printf()`, ký tự `%` được dùng để đánh dấu sự bắt đầu của một **conversion specification** (bộ định dạng chuyển đổi), sau đó là các ký tự như `d`, `f`, `s`, v.v. để chỉ định kiểu dữ liệu.

- Ví dụ: Trong `printf("Gia tri: %d", 42);`, `%d` là conversion specification để in số nguyên.

5. **getchar()** is a function without any arguments. (T/F)

Trả lời: **TRUE**.

Giải thích:

- Hàm `getchar()` trong C là một hàm không có tham số (arguments). Nó đọc một ký tự từ luồng nhập chuẩn (stdin) và trả về giá trị kiểu `int` (thường là mã ASCII của ký tự hoặc `EOF` nếu có lỗi).

- Cú pháp: `int c = getchar();` – không cần truyền tham số nào.

6. **_____** is a temporary storage area in memory.

Trả lời: **Buffer** is a temporary storage area in memory.

Đáp án đúng: C. Buffer.

Giải thích:

- Bộ đệm (buffer) là một vùng lưu trữ tạm thời trong bộ nhớ, được sử dụng để lưu dữ liệu trong quá trình đọc/ghi, nhằm tối ưu hóa hiệu suất I/O.

- Ví dụ: Khi dùng `printf()`, dữ liệu có thể được lưu trong bộ đệm trước khi xuất ra màn hình, và `fflush()` có thể được dùng để xóa bộ đệm.

7. **Escape sequence can be placed outside the control string in printf().** (T/F)

Trả lời: **FALSE**.

Giải thích:

- Chuỗi thoát (escape sequence) như `\n`, `\t`, `\b`, v.v. phải được đặt **bên trong** chuỗi điều khiển (control string) của `printf()` để được xử lý đúng.

- Ví dụ: `printf("Xin chào\n");` – `\n` nằm trong chuỗi điều khiển, tạo dòng mới.

- Nếu đặt bên ngoài chuỗi điều khiển, như `printf("Xin chào")\n;`, thì `\n` không có ý nghĩa trong ngữ cảnh này và sẽ gây lỗi biên dịch hoặc không hoạt động như mong muốn.

Tóm tắt câu trả lời:

1. The formatted I/O functions are **printf()** and **scanf()** (A).
2. `scanf()` uses **pointers** to variables rather than variable names (B).
3. **Format specifiers** specify the form by which the values of the variables are to be input/printed (B).
4. **%%** is used by the `printf()` function to identify conversion specifications (A).
5. `getchar()` is a function without any arguments: **TRUE**.
6. **Buffer** is a temporary storage area in memory (C).
7. Escape sequence can be placed outside the control string in `printf()`: **FALSE**.

Dưới đây là câu trả lời cho 5 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. ****_____ statements enable us to change the flow of a program.****

****Trả lời**:** ****Conditional**** statements enable us to change the flow of a program.

****Đáp án đúng**:** A. Conditional.

****Giải thích**:**

- Câu lệnh điều kiện (conditional statements) trong C, như ``if``, ``else if``, ``switch``, cho phép thay đổi luồng thực thi của chương trình dựa trên điều kiện.

- Ví dụ: ``if (x > 0) { printf("Duong"); } else { printf("Am"); }`` – luồng chương trình thay đổi tùy thuộc vào giá trị của ``x``.

2. ****The else statement is optional. (T/F)****

****Trả lời**:** ****TRUE****.

****Giải thích**:**

- Trong C, câu lệnh ``else`` trong cấu trúc ``if-else`` là ****tùy chọn****. Bạn có thể chỉ sử dụng ``if`` mà không cần ``else``.

- Ví dụ: ``if (x > 0) { printf("Duong"); }`` – không có ``else``, chương trình vẫn hợp lệ. Nếu có ``else``, nó sẽ xử lý trường hợp điều kiện trong ``if`` là sai: ``if (x > 0) { printf("Duong"); } else { printf("Khong duong"); }``.

3. ****A _____ is an if statement, which is placed within another if or else.****

****Trả lời**:** A ****nested if**** is an if statement, which is placed within another if or else.

****Đáp án đúng**:** B. Nested if.

****Giải thích**:**

- Câu lệnh ``if`` lồng nhau (nested if) là một câu lệnh ``if`` được đặt bên trong một câu lệnh ``if`` hoặc ``else`` khác.

- Ví dụ:

```
```\nc\nif (x > 0) {\n    if (x < 10) { // Nested if\n        printf("x nam trong khoang 0 den 10");\n    }\n}\n```\n
```

- Ở đây, ``if (x < 10)`` được lồng trong ``if (x > 0)``.

4. **\*\*The \_\_\_\_\_ statement is a multi-way decision maker that tests the value of an expression against a list of integer or character constants.\*\***

**\*\*Trả lời\*\*:** The **\*\*switch\*\*** statement is a multi-way decision maker that tests the value of an expression against a list of integer or character constants.

**\*\*Đáp án đúng\*\*:** C. Switch.

**\*\*Giải thích\*\*:**

- Câu lệnh ``switch`` trong C là một cơ chế ra quyết định đa hướng (multi-way decision maker). Nó kiểm tra giá trị của một biểu thức (expression) và so sánh với các hằng số kiểu nguyên (``int``) hoặc ký tự (``char``), sau đó thực thi khối lệnh tương ứng.

- Ví dụ:

```
```\nc\nswitch (grade) {\n
```

```

    case 'A': printf("Xuat sac"); break;
    case 'B': printf("Tot"); break;
    default: printf("Khong xep loai");
}
...

```

- Ở đây, `switch` kiểm tra giá trị của `grade` và so sánh với các hằng ký tự (`'A'`, `'B'`).

5. **if (expression)**

statement 1

else

statement 2

Which statement will be executed when expression is false?

Trả lời: **Statement 2** will be executed when expression is false.

Đáp án đúng: B. Statement 2.

Giải thích:

- Trong cấu trúc `if-else`, nếu biểu thức (`expression`) trong `if` là **true**, thì `statement 1` sẽ được thực thi. Nếu biểu thức là **false**, thì `statement 2` trong phần `else` sẽ được thực thi.

- Ví dụ:

```

...c
int x = 0;
if (x > 0) {
    printf("Duong"); // statement 1
} else {
    printf("Khong duong"); // statement 2
}
...

```

- Vì `x > 0` là **false** (do `x = 0`), `statement 2` (`printf("Khong duong");`) sẽ được thực thi.

Tóm tắt câu trả lời:

1. **Conditional** statements enable us to change the flow of a program (A).
2. The else statement is optional: **TRUE**.
3. A **nested if** is an if statement, which is placed within another if or else (B).
4. The **switch** statement is a multi-way decision maker that tests the value of an expression (C).
5. **Statement 2** will be executed when expression is false (B).

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

Dưới đây là câu trả lời cho 6 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **_____** allows a set of instructions to be performed until a certain condition is reached.

Trả lời: **Loop** allows a set of instructions to be performed until a certain condition is reached.

Đáp án đúng: A. Loop.

****Giải thích**:**

- Vòng lặp (loop) trong lập trình cho phép một tập hợp các câu lệnh được thực thi lặp lại cho đến khi một điều kiện cụ thể được thỏa mãn.

- Ví dụ: Trong C, vòng lặp `for`, `while`, và `do-while` đều là các loại vòng lặp. Ví dụ: `for (int i = 0; i < 5; i++) { printf("%d ", i); }` lặp lại việc in từ 0 đến 4.

2. ****_____ loops check the condition at the top of the loop which means the loop code is not executed, if the condition is false at the start.****

****Trả lời**:** ****while**** loops check the condition at the top of the loop which means the loop code is not executed, if the condition is false at the start.

****Đáp án đúng**:** A. while loop.

****Giải thích**:**

- Vòng lặp `while` kiểm tra điều kiện ở ****đầu vòng lặp****. Nếu điều kiện sai ngay từ đầu, khối mã bên trong vòng lặp sẽ không được thực thi.

- Ví dụ: `int i = 10; while (i < 5) { printf("%d ", i); i++; }` – vì `i < 5` là `false` ngay từ đầu (`i = 10`), khối mã không chạy.

- So sánh với `do-while`, vòng lặp này kiểm tra điều kiện ở cuối, nên khối mã chạy ít nhất một lần.

3. ****A _____ is used to separate the three parts of the expression in a for loop.****

****Trả lời**:** A ****semicolon**** is used to separate the three parts of the expression in a for loop.

****Đáp án đúng**:** B. semicolon.

****Giải thích**:**

- Trong vòng lặp `for` của C, cú pháp là: `for (khởi\_tạo; điều\_kiện; cập\_nhật) { mã; }`. Ba phần (khởi tạo, điều kiện, cập nhật) được phân tách bằng dấu chấm phẩy (;).

- Ví dụ: `for (int i = 0; i < 5; i++) { printf("%d ", i); }` – dấu `;` tách `int i = 0`, `i < 5`, và `i++`.

4. ****The _____ loop checks its condition at the end of the loop, that is after the loop has been executed.****

****Trả lời**:** The ****do...while**** loop checks its condition at the end of the loop, that is after the loop has been executed.

****Đáp án đúng**:** C. do...while loop.

****Giải thích**:**

- Vòng lặp `do-while` kiểm tra điều kiện ở ****cuối vòng lặp****, nghĩa là khối mã bên trong sẽ được thực thi ít nhất một lần trước khi điều kiện được kiểm tra.

- Ví dụ:

```
``c
int i = 10;
do {
    printf("%d ", i);
    i++;
} while (i < 5);
```
```

- Ở đây, dù `i < 5` là `false` ngay từ đầu (`i = 10`), khối mã vẫn chạy một lần, in `10`.

5. **\*\*The \_\_\_\_\_ statement causes execution to return to the point at which the call to the function was made.\*\***

**\*\*Trả lời\*\*:** The **\*\*return\*\*** statement causes execution to return to the point at which the call to the function was made.

**\*\*Đáp án đúng\*\*:** B. return.

**\*\*Giải thích\*\*:**

- Câu lệnh ``return`` trong C được sử dụng để kết thúc một hàm và trả về một giá trị (nếu có) cho nơi gọi hàm đó. Nó cũng chuyển luồng thực thi trở lại điểm gọi hàm.

- Ví dụ:

```
```c
int sum(int a, int b) {
    return a + b; // Trả về tổng và thoát hàm
}
int main() {
    int result = sum(3, 4); // result = 7
    return 0;
}
...

```

- ``return a + b;`` trả về giá trị ``7`` và thoát hàm ``sum``.

6. ****The _____ statement violates the rules of a strictly structured programming language.****

****Trả lời**:** The ****goto**** statement violates the rules of a strictly structured programming language.

****Đáp án đúng**:** C. goto.

****Giải thích**:**

- Câu lệnh ``goto`` trong C cho phép nhảy trực tiếp đến một nhãn (label) trong chương trình, phá vỡ luồng điều khiển có cấu trúc (structured programming).

- Lập trình có cấu trúc khuyến khích sử dụng các cấu trúc điều khiển như ``if``, ``for``, ``while``, vì chúng rõ ràng và dễ bảo trì. ``goto`` thường gây khó khăn trong việc đọc và gỡ lỗi mã, nên bị coi là vi phạm nguyên tắc lập trình có cấu trúc.

- Ví dụ:

```
```c
if (x < 0) goto error;
error: printf("Loi!");
...

```

- ``goto`` nhảy trực tiếp đến nhãn ``error``, làm luồng chương trình khó theo dõi.

---

**### Tóm tắt câu trả lời:**

1. **\*\*Loop\*\*** allows a set of instructions to be performed until a certain condition is reached (A).
2. **\*\*while\*\*** loops check the condition at the top of the loop (A).
3. A **\*\*semicolon\*\*** is used to separate the three parts of the expression in a for loop (B).
4. The **\*\*do...while\*\*** loop checks its condition at the end of the loop (C).
5. The **\*\*return\*\*** statement causes execution to return to the point at which the call to the function was made (B).
6. The **\*\*goto\*\*** statement violates the rules of a strictly structured programming language (C).

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

The \_\_\_\_\_ function causes immediate termination of the program and control is transferred back to the operating system

A. exit

Dưới đây là câu trả lời cho 8 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **An \_\_\_\_\_ is a collection of data elements of the same type that are referred by a common name.**

**Trả lời:** An **array** is a collection of data elements of the same type that are referred by a common name.

**Đáp án đúng:** B. Array.

**Giải thích:**

- Mảng (array) trong C là một tập hợp các phần tử có cùng kiểu dữ liệu, được tham chiếu bằng một tên chung. Mỗi phần tử trong mảng được truy cập thông qua chỉ số (index).

- Ví dụ: `int arr[5] = {1, 2, 3, 4, 5};` – `arr` là một mảng chứa 5 số nguyên.

2. **Each member in an array is identified by the unique \_\_\_\_\_ or \_\_\_\_\_ assigned to it.**

**Trả lời:** Each member in an array is identified by the unique **index** or **subscript** assigned to it.

**Đáp án đúng:** B. Subscript, Index.

**Giải thích:**

- Trong mảng, mỗi phần tử được xác định bởi một **chỉ số** (index) hoặc **subscript** duy nhất, bắt đầu từ 0.

- Ví dụ: Trong `int arr[3] = {10, 20, 30};`, `arr[0]` là 10, `arr[1]` là 20, `arr[2]` là 30. Ở đây, `0`, `1`, `2` là chỉ số (index/subscript).

3. **An array name and a variable name can be the same. (T/F)**

**Trả lời:** **FALSE**.

**Giải thích:**

- Trong C, tên mảng và tên biến là các **định danh** (identifier), và chúng không được trùng nhau trong cùng phạm vi (scope). Nếu trùng, chương trình sẽ báo lỗi biên dịch vì xung đột tên.

- Ví dụ:

```
``c
int x = 5;
int x[5]; // Lỗi: Trùng tên 'x'
```
```

- Tên mảng và biến phải là duy nhất trong cùng phạm vi.

4. **Each element of an array cannot be used where a variable is allowed or required. (T/F)**

Trả lời: **FALSE**.

Giải thích:

- Mỗi phần tử của mảng có thể được sử dụng ở bất kỳ nơi nào mà một biến cùng kiểu được phép hoặc yêu cầu. Phần tử mảng thực chất hoạt động như một biến của kiểu dữ liệu đó.

- Ví dụ:

```
```c
int arr[3] = {1, 2, 3};
arr[0] = 10; // Gán giá trị như biến
printf("%d", arr[0]); // Sử dụng như biến
```
```

- `arr[0]` có thể được sử dụng như một biến kiểu `int`.

5. **Two arrays, even if they are of the same type and size, cannot be tested for _____.**

Trả lời: Two arrays, even if they are of the same type and size, cannot be tested for **equality**.

Đáp án đúng: C. Equality.

Giải thích:

- Trong C, không thể so sánh trực tiếp hai mảng bằng toán tử `==` để kiểm tra xem chúng có bằng nhau hay không, ngay cả khi chúng cùng kiểu và kích thước.

- Lý do: Tên mảng là con trỏ đến phần tử đầu tiên, và toán tử `==` sẽ so sánh địa chỉ bộ nhớ, không phải giá trị các phần tử.

- Ví dụ:

```
```c
int arr1[3] = {1, 2, 3};
int arr2[3] = {1, 2, 3};
if (arr1 == arr2) // Sai: So sánh địa chỉ, không phải giá trị
```
```

- Để so sánh, bạn cần duyệt qua từng phần tử.

6. **String can be defined as a character type array, which is terminated by a _____ character.**

Trả lời: String can be defined as a character type array, which is terminated by a **NULL** character.

Đáp án đúng: D. NULL.

Giải thích:

- Trong C, chuỗi (string) là một mảng ký tự (`char`) được kết thúc bằng ký tự `NULL` (ký hiệu `\0`). Ký tự này báo hiệu điểm kết thúc của chuỗi.

- Ví dụ: `char str[] = "Hello";` – Chuỗi này thực chất là `{ 'H', 'e', 'l', 'l', 'o', '\0' }`, với `\0` là ký tự NULL.

7. **Arrays can have more than one dimension. (T/F)**

Trả lời: **TRUE**.

Giải thích:

- Trong C, mảng có thể có nhiều chiều (multidimensional array), chẳng hạn như mảng 2 chiều, 3 chiều, v.v.

- Ví dụ:

```
```c
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}}; // Mảng 2 chiều
```
```

- `matrix` là mảng 2 chiều với 2 hàng và 3 cột. Bạn có thể khai báo mảng với nhiều chiều hơn nếu cần.

8. **The comparison of two strings is done with the help of _____ whereas the interchanging is done by _____.**

Trả lời: The comparison of two strings is done with the help of **strcmp** whereas the interchanging is done by **strcpy**.

Đáp án đúng: A. strcmp, strcpy.

Giải thích:

- So sánh chuỗi: Hàm `strcmp()` (trong `<string.h>`) được dùng để so sánh hai chuỗi. Nó trả về 0 nếu chuỗi bằng nhau, giá trị âm nếu chuỗi 1 nhỏ hơn, và giá trị dương nếu chuỗi 1 lớn hơn. Ví dụ: `strcmp("abc", "abc")` trả về 0.

- Hoán đổi chuỗi: Hàm `strcpy()` (cũng trong `<string.h>`) được dùng để sao chép một chuỗi vào một chuỗi khác, thường dùng trong hoán đổi. Ví dụ:

```
```\nc
char str1[10] = "Hello";
char str2[10] = "World";
char temp[10];
strcpy(temp, str1); // Sao chép str1 vào temp
strcpy(str1, str2); // Sao chép str2 vào str1
strcpy(str2, temp); // Sao chép temp vào str2
```
```

Tóm tắt câu trả lời:

1. An **array** is a collection of data elements of the same type (B).
2. Each member in an array is identified by the unique **subscript** or **index** (B).
3. An array name and a variable name can be the same: **FALSE**.
4. Each element of an array cannot be used where a variable is allowed or required: **FALSE**.
5. Two arrays, even if they are of the same type and size, cannot be tested for **equality** (C).
6. String can be defined as a character type array, which is terminated by a **NULL** character (D).
7. Arrays can have more than one dimension: **TRUE**.
8. The comparison of two strings is done with the help of **strcmp** whereas the interchanging is done by **strcpy** (A).

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

Dưới đây là câu trả lời cho 7 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **A _____ provides a way of accessing a variable without referring to the variable directly.**

Trả lời: A **pointer** provides a way of accessing a variable without referring to the variable directly.

Đáp án đúng: B. Pointer.

****Giải thích**:**

- Con trỏ (pointer) trong C là một biến lưu trữ địa chỉ bộ nhớ của một biến khác. Nó cho phép truy cập gián tiếp vào biến thông qua địa chỉ mà không cần tham chiếu trực tiếp đến tên biến.

- Ví dụ:

```
```c
int x = 10;
int *ptr = &x; // ptr trỏ đến x
printf("%d", *ptr); // Truy cập x gián tiếp, in ra 10
```
```

2. ****Pointers cannot point to arrays. (T/F)****

****Trả lời**:** ****FALSE****.

****Giải thích**:**

- Con trỏ có thể trỏ đến mảng. Trong C, tên mảng thực chất là một con trỏ đến phần tử đầu tiên của mảng.

- Ví dụ:

```
```c
int arr[3] = {1, 2, 3};
int *ptr = arr; // ptr trỏ đến arr[0]
printf("%d", *(ptr + 1)); // In ra 2 (arr[1])
```
```

- Con trỏ thường được dùng để duyệt mảng một cách hiệu quả.

3. ****The _____ type of pointer defines what type of variable the pointer can point to.****

****Trả lời**:** The ****content**** type of pointer defines what type of variable the pointer can point to.

****Đáp án đúng**:** A. Content.

****Giải thích**:**

- Kiểu nội dung (content type) của con trỏ xác định kiểu dữ liệu của biến mà con trỏ có thể trỏ đến. Ví dụ, `int *ptr` là con trỏ trỏ đến biến kiểu `int`, `float *ptr` trỏ đến biến kiểu `float`.

- Điều này đảm bảo con trỏ giải tham chiếu (`*`) đúng kiểu dữ liệu. Ví dụ:

```
```c
int x = 5;
int *ptr = &x; // Kiểu nội dung là int
```
```

4. ****The two special operators used with pointers are _____ and _____.****

****Trả lời**:** The two special operators used with pointers are `*` and `&`.

****Đáp án đúng**:** C. `*` and `&`.

****Giải thích**:**

- Hai toán tử đặc biệt dùng với con trỏ trong C là:

- `*` (toán tử giải tham chiếu): Truy cập giá trị tại địa chỉ mà con trỏ trỏ đến. Ví dụ: `*ptr` truy cập giá trị của biến mà `ptr` trỏ đến.

- `&` (toán tử lấy địa chỉ): Lấy địa chỉ bộ nhớ của một biến. Ví dụ: `&x` trả về địa chỉ của `x`.

- Ví dụ:

```
```c
```

```
int x = 10;
int *ptr = &x; // & lấy địa chỉ của x
printf("%d", *ptr); // * truy cập giá trị, in ra 10
...
```

5. **Pointers \_\_\_\_\_ and \_\_\_\_\_ are the only operations, which can be performed on pointers.**

**Trả lời:** Pointers **addition** and **subtraction** are the only operations, which can be performed on pointers.

**Đáp án đúng:** A. Addition, Subtraction.

**Giải thích:**

- Trong C, các phép toán có thể thực hiện trên con trỏ bao gồm **cộng** (addition) và **trừ** (subtraction) để di chuyển con trỏ trong bộ nhớ.

- Ví dụ:

```
```c
int arr[3] = {1, 2, 3};
int *ptr = arr;
ptr = ptr + 1; // Di chuyển con trỏ đến phần tử tiếp theo (arr[1])
printf("%d", *ptr); // In ra 2
...
```

- Các phép toán khác như nhân (*) hoặc chia (/) không được phép trực tiếp trên con trỏ.

6. **Pointer types can be compared only if both these variables are pointing to variables of different types. (T/F)**

Trả lời: **FALSE**.

Giải thích:

- Con trỏ có thể được so sánh (dùng ==, !=, <, >, v.v.) nếu chúng trỏ đến các biến **cùng kiểu dữ liệu**, không phải khác kiểu. So sánh con trỏ thường được dùng để kiểm tra vị trí trong bộ nhớ, đặc biệt khi làm việc với mảng.

- Ví dụ:

```
```c
int arr[3] = {1, 2, 3};
int *p1 = arr;
int *p2 = arr + 1;
if (p1 < p2) printf("p1 truoac p2"); // Hợp lệ
...
```

- So sánh con trỏ trỏ đến các kiểu dữ liệu khác nhau là không có ý nghĩa và có thể gây lỗi.

7. **The allocation of memory in this manner, that is, as and when required in a program is known as \_\_\_\_\_ Memory Allocation.**

**Trả lời:** The allocation of memory in this manner, that is, as and when required in a program is known as **Dynamic** Memory Allocation.

**Đáp án đúng:** A. Dynamic Memory Allocation.

**Giải thích:**

- Cấp phát bộ nhớ động (dynamic memory allocation) là cách cấp phát bộ nhớ trong thời gian chạy chương trình, khi cần thiết, sử dụng các hàm như malloc(), calloc(), realloc(), và giải phóng bằng free().

- Ví dụ:

```
```c
```

```
int *ptr = (int *)malloc(5 * sizeof(int)); // Cấp phát động cho 5 số nguyên
```

```
free(ptr); // Giải phóng
```

```
```
```

- Điều này khác với cấp phát tĩnh (static), nơi bộ nhớ được cấp phát tại thời điểm biên dịch.

---

### Tóm tắt câu trả lời:

1. A **pointer** provides a way of accessing a variable without referring to the variable directly (B).
2. Pointers cannot point to arrays: **FALSE**.
3. The **content** type of pointer defines what type of variable the pointer can point to (A).
4. The two special operators used with pointers are ```` and `&` (C).
5. Pointers **addition** and **subtraction** are the only operations, which can be performed on pointers (A).
6. Pointer types can be compared only if both these variables are pointing to variables of different types: **FALSE**.
7. The allocation of memory in this manner, that is, as and when required in a program is known as **Dynamic** Memory Allocation (A).

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

Dưới đây là câu trả lời cho 10 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **A \_\_\_\_\_ is a self-contained program segment that carries out a specific, well-defined task.**

**Trả lời:** A **function** is a self-contained program segment that carries out a specific, well-defined task.

**Giải thích:**

- Hàm (function) trong C là một đoạn chương trình độc lập, được thiết kế để thực hiện một nhiệm vụ cụ thể, có thể tái sử dụng.

- Ví dụ: `int sum(int a, int b) { return a + b; }` – hàm `sum` thực hiện nhiệm vụ tính tổng hai số.

2. **Arguments appearing in the parentheses are termed as \_\_\_\_\_.**

**Trả lời:** Arguments appearing in the parentheses are termed as **parameters**.

**Giải thích:**

- Các đối số (arguments) xuất hiện trong dấu ngoặc đơn của khai báo hàm được gọi là **tham số** (parameters). Chúng là các biến được sử dụng để nhận giá trị từ nơi gọi hàm.

- Ví dụ: Trong `void myFunction(int x, int y)`, `x` và `y` là tham số.

3. **If the return is ignored, control passes to the calling program when the closing braces of the code block are encountered. This is termed as \_\_\_\_\_.**

**Trả lời:** If the return is ignored, control passes to the calling program when the closing braces of the code block are encountered. This is termed as **implicit return**.



**\*\*Giải thích\*\*:**

- Nếu một hàm không có câu lệnh `return` (hoặc hàm có kiểu trả về `void`), điều khiển sẽ tự động quay lại chương trình gọi khi gặp dấu ngoặc đóng `}` của hàm. Đây được gọi là **\*\*trả về ngầm\*\*** (implicit return).

- Ví dụ:

```
```c
void printHello() {
    printf("Hello");
} // Khi gặp }, điều khiển tự động quay lại nơi gọi
```
```

4. **\*\*The function, which calls another function, is known as the \_\_\_\_\_, and the function, which is being called, is known as the \_\_\_\_\_.\*\***

**\*\*Trả lời\*\*:** The function, which calls another function, is known as the **\*\*caller\*\***, and the function, which is being called, is known as the **\*\*callee\*\***.

**\*\*Giải thích\*\*:**

- Hàm gọi một hàm khác được gọi là **\*\*caller\*\*** (hàm gọi), và hàm được gọi được gọi là **\*\*callee\*\*** (hàm bị gọi).

- Ví dụ:

```
```c
void callee() { printf("Goi toi"); }
void caller() { callee(); } // caller gọi callee
```
```

5. **\*\*A \_\_\_\_\_ is the function declaration that specifies the data types of the arguments.\*\***

**\*\*Trả lời\*\*:** A **\*\*prototype\*\*** is the function declaration that specifies the data types of the arguments.

**\*\*Giải thích\*\*:**

- Nguyên mẫu hàm (function prototype) là khai báo hàm trước khi sử dụng, chỉ rõ kiểu trả về và kiểu dữ liệu của các tham số, giúp trình biên dịch kiểm tra tính hợp lệ.

- Ví dụ: `int sum(int, int);` – đây là nguyên mẫu của hàm `sum` nhận hai tham số kiểu `int` và trả về `int`.

6. **\*\*\_\_\_\_\_ can be referred to only by statements that are inside the code block, which declares them.\*\***

**\*\*Trả lời\*\*:** **\*\*Local variables\*\*** can be referred to only by statements that are inside the code block, which declares them.

**\*\*Giải thích\*\*:**

- Biến cục bộ (local variables) là các biến được khai báo bên trong một khối mã (block), chẳng hạn như bên trong một hàm. Chúng chỉ có thể được truy cập bởi các câu lệnh trong cùng khối mã đó.

- Ví dụ:

```
```c
void myFunction() {
    int x = 5; // Biến cục bộ
    printf("%d", x); // Hợp lệ
}
// printf("%d", x); // Lỗi: x không tồn tại ngoài hàm
```
```

...

7. **\*\*\_\_\_\_\_ are visible to the entire program, and can be used by any piece of code.\*\***

**\*\*Trả lời\*\*:** **\*\*Global variables\*\*** are visible to the entire program, and can be used by any piece of code.

**\*\*Giải thích\*\*:**

- Biến toàn cục (global variables) là các biến được khai báo bên ngoài tất cả các hàm, có phạm vi toàn chương trình và có thể được truy cập bởi bất kỳ đoạn mã nào.

- Ví dụ:

```
```c
int globalVar = 10; // Biến toàn cục
void func1() { printf("%d", globalVar); } // Hợp lệ
void func2() { printf("%d", globalVar); } // Hợp lệ
```
```

8. **\*\*A piece of code governs whether one piece of code knows about or has access to another piece of code or data.\*\***

**\*\*Trả lời\*\*:** A piece of code governs whether one piece of code knows about or has access to another piece of code or data – this is **\*\*scope\*\***.

**\*\*Giải thích\*\*:**

- Phạm vi (scope) xác định nơi mà một biến hoặc hàm có thể được truy cập trong chương trình. Có các loại phạm vi như phạm vi cục bộ (local scope), phạm vi toàn cục (global scope), và phạm vi khối (block scope).

- Ví dụ: Biến cục bộ chỉ có phạm vi trong hàm, còn biến toàn cục có phạm vi toàn chương trình.

9. **\*\*Arguments are said to be passed \_\_\_\_\_ when the value of the variables are passed to the called function.\*\***

**\*\*Trả lời\*\*:** Arguments are said to be passed **\*\*by value\*\*** when the value of the variables are passed to the called function.

**\*\*Giải thích\*\*:**

- Truyền tham số theo giá trị (pass by value) nghĩa là giá trị của biến được sao chép và truyền vào hàm. Thay đổi trong hàm không ảnh hưởng đến biến gốc.

- Ví dụ:

```
```c
void change(int x) { x = 20; }
int main() {
    int a = 5;
    change(a); // Truyền giá trị của a
    printf("%d", a); // Vẫn in 5, không đổi
    return 0;
}
```
```

10. **\*\*In \_\_\_\_\_, the function is allowed access to the actual memory location of the argument.\*\***

**\*\*Trả lời\*\*:** In **\*\*pass by reference\*\***, the function is allowed access to the actual memory location of the argument.

**\*\*Giải thích\*\*:**

- Truyền tham chiếu (pass by reference) nghĩa là hàm nhận địa chỉ của biến (thông qua con trỏ), cho phép hàm truy cập và thay đổi trực tiếp giá trị tại địa chỉ đó.

- Ví dụ:

```
```c
void change(int *x) { *x = 20; }
int main() {
    int a = 5;
    change(&a); // Truyền địa chỉ của a
    printf("%d", a); // In ra 20, giá trị đã đổi
    return 0;
}
```
```

---

### Tóm tắt câu trả lời:

1. A **\*\*function\*\*** is a self-contained program segment that carries out a specific, well-defined task.
2. Arguments appearing in the parentheses are termed as **\*\*parameters\*\***.
3. If the return is ignored, control passes to the calling program when the closing braces of the code block are encountered. This is termed as **\*\*implicit return\*\***.
4. The function, which calls another function, is known as the **\*\*caller\*\***, and the function, which is being called, is known as the **\*\*callee\*\***.
5. A **\*\*prototype\*\*** is the function declaration that specifies the data types of the arguments.
6. **\*\*Local variables\*\*** can be referred to only by statements that are inside the code block, which declares them.
7. **\*\*Global variables\*\*** are visible to the entire program, and can be used by any piece of code.
8. A piece of code governs whether one piece of code knows about or has access to another piece of code or data – this is **\*\*scope\*\***.
9. Arguments are said to be passed **\*\*by value\*\*** when the value of the variables are passed to the called function.
10. In **\*\*pass by reference\*\***, the function is allowed access to the actual memory location of the argument.

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

Dưới đây là câu trả lời cho 8 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **\*\*Strings are terminated by the \_\_\_\_\_ character.\*\***

**\*\*Trả lời\*\*:** Strings are terminated by the **\*\*NULL\*\*** character.

**\*\*Giải thích\*\*:**

- Trong C, chuỗi (string) là một mảng ký tự (`char`) và được kết thúc bởi ký tự `NULL` (ký hiệu `\0`). Ký tự này đánh dấu điểm kết thúc của chuỗi.

- Ví dụ: `char str[] = "Hello";` – Chuỗi này thực chất là `{ 'H', 'e', 'l', 'l', 'o', '\0' }`, với `\0` là ký tự NULL.

2. **\*\*The number of characters that can be input into char arr[15] is \_\_\_\_\_.\*\***

**\*\*Trả lời\*\*:** The number of characters that can be input into char arr[15] is **\*\*14\*\***.

**\*\*Giải thích\*\*:**

- Mảng `char arr[15]` có kích thước 15 phần tử, nhưng khi dùng để lưu chuỗi, cần dành 1 phần tử cho ký tự kết thúc `'\0'`. Vì vậy, số ký tự tối đa có thể nhập là **\*\*15 - 1 = 14\*\***.

- Ví dụ: `char arr[15] = "12345678901234";` – 14 ký tự cộng với `'\0'` vừa đủ.

3. **\*\*Modification of the string pointer can lead to data loss. (TRUE / FALSE)\*\***

**\*\*Trả lời\*\*:** **\*\*TRUE\*\***.

**\*\*Giải thích\*\*:**

- Con trỏ chuỗi (string pointer) trỏ đến địa chỉ bộ nhớ của chuỗi. Nếu sửa đổi con trỏ (ví dụ: di chuyển con trỏ hoặc ghi đè chuỗi) không cẩn thận, có thể dẫn đến mất dữ liệu hoặc truy cập bộ nhớ không hợp lệ.

- Ví dụ:

```
```c
char str[] = "Hello";
char *ptr = str;
ptr++; // Con trỏ di chuyển, mất ký tự 'H'
printf("%s", ptr); // In ra "ello"
```
```

- Nếu chuỗi là hằng (string literal) như `char *ptr = "Hello";`, việc sửa đổi trực tiếp có thể gây lỗi (undefined behavior).

4. **\*\*The character used to print a new line in printf() is \_\_\_\_\_.\*\***

**\*\*Trả lời\*\*:** The character used to print a new line in printf() is **\*\*'\n'\*\***.

**\*\*Giải thích\*\*:**

- Trong `printf()`, ký tự thoát `'\n'` (newline) được sử dụng để in một dòng mới, di chuyển con trỏ màn hình xuống dòng tiếp theo.

- Ví dụ: `printf("Hello\nWorld");` – in ra:

```
```
Hello
World
```
```

5. **\*\*To use the strcat() function, the \_\_\_\_\_ header file must be included in the program.\*\***

**\*\*Trả lời\*\*:** To use the strcat() function, the **\*\*string.h\*\*** header file must be included in the program.

**\*\*Giải thích\*\*:**

- Hàm `strcat()` (nối chuỗi) được định nghĩa trong tệp tiêu đề `<string.h>`. Để sử dụng, chương trình cần bao gồm dòng `#include <string.h>`.

- Ví dụ:

```
```c
#include <string.h>
char str1[20] = "Hello";
char str2[] = "World";
strcat(str1, str2); // Nối str2 vào str1
```
```

6. **\*\*Two pointers can be compared only if both these variables are pointing to variables of different types. (TRUE / FALSE)\*\***

**\*\*Trả lời\*\*:** **\*\*FALSE\*\***.

**\*\*Giải thích\*\*:**

- Hai con trỏ có thể được so sánh (dùng `'=='`, `'!='`, `'<'`, `'>'`, v.v.) nếu chúng trỏ đến các biến **\*\*cùng kiểu dữ liệu\*\***, không phải khác kiểu. So sánh thường được dùng để kiểm tra vị trí bộ nhớ, đặc biệt với mảng.

- Ví dụ:

```
```c
int arr[3] = {1, 2, 3};
int *p1 = arr;
int *p2 = arr + 1;
if (p1 < p2) printf("p1 trước p2"); // Hợp lệ
```
```

- So sánh con trỏ trỏ đến các kiểu dữ liệu khác nhau là không có ý nghĩa và có thể gây lỗi.

7. **\*\*strcmp() returns \_\_\_\_\_ if the two strings are identical.\*\***

**\*\*Trả lời\*\*:** strcmp() returns **\*\*0\*\*** if the two strings are identical.

**\*\*Giải thích\*\*:**

- Hàm `'strcmp()'` (trong `'<string.h>'`) so sánh hai chuỗi và trả về:

- `'0'` nếu hai chuỗi giống nhau.
- Giá trị âm nếu chuỗi 1 nhỏ hơn chuỗi 2 (theo thứ tự từ điển).
- Giá trị dương nếu chuỗi 1 lớn hơn chuỗi 2.
- Ví dụ: `'strcmp("abc", "abc")'` trả về `'0'` vì hai chuỗi giống nhau.

8. **\*\*When an array is passed to a function, only its \_\_\_\_\_ is passed.\*\***

**\*\*Trả lời\*\*:** When an array is passed to a function, only its **\*\*address\*\*** is passed.

**\*\*Giải thích\*\*:**

- Trong C, khi một mảng được truyền vào hàm, thực chất chỉ **\*\*địa chỉ của phần tử đầu tiên\*\*** của mảng được truyền (tên mảng là con trỏ đến phần tử đầu tiên). Điều này gọi là truyền theo tham chiếu (pass by reference).

- Ví dụ:

```
```c
void printArray(int arr[], int size) {
    printf("%d", arr[0]);
}

int main() {
    int arr[3] = {1, 2, 3};
    printArray(arr, 3); // Truyền địa chỉ của arr
    return 0;
}
```
```

---

**### Tóm tắt câu trả lời:**

1. Strings are terminated by the **\*\*NULL\*\*** character.
2. The number of characters that can be input into `char arr[15]` is **\*\*14\*\***.

3. Modification of the string pointer can lead to data loss: **\*\*TRUE\*\***.
4. The character used to print a new line in printf() is **\*\*\n\*\***.
5. To use the strcat() function, the **\*\*string.h\*\*** header file must be included in the program.
6. Two pointers can be compared only if both these variables are pointing to variables of different types: **\*\*FALSE\*\***.
7. strcmp() returns **\*\*0\*\*** if the two strings are identical.
8. When an array is passed to a function, only its **\*\*address\*\*** is passed.

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

Dưới đây là câu trả lời cho 7 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **\*\*A \_\_\_\_\_ data type groups together a number of data items, which need not be of the same type.\*\***

**\*\*Trả lời\*\***: A **\*\*structure\*\*** data type groups together a number of data items, which need not be of the same type.

**\*\*Giải thích\*\***:

- Cấu trúc (structure) trong C là một kiểu dữ liệu cho phép nhóm nhiều mục dữ liệu (data items) có thể thuộc các kiểu dữ liệu khác nhau dưới một tên chung.

- Ví dụ:

```
``c
struct Person {
 char name[20];
 int age;
 float height;
};
...

```

- Ở đây, `name` (chuỗi), `age` (số nguyên), và `height` (số thực) thuộc các kiểu khác nhau nhưng được nhóm lại trong cấu trúc `Person`.

2. **\*\*Individual structure elements are referenced through the use of the \_\_\_\_\_.\*\***

**\*\*Trả lời\*\***: Individual structure elements are referenced through the use of the **\*\*dot operator\*\***.

**\*\*Giải thích\*\***:

- Toán tử chấm (dot operator, `.`) được sử dụng để truy cập các phần tử (members) của một biến cấu trúc.

- Ví dụ:

```
``c
struct Person p;
p.age = 25; // Truy cập phần tử age bằng toán tử .
strcpy(p.name, "Grok");
...

```

3. **\*\*Values of one structure variable can be assigned to another variable of the same type using a simple assignment statement. (TRUE / FALSE)\*\***

**\*\*Trả lời\*\***: **\*\*TRUE\*\***.

**\*\*Giải thích\*\***:

- Trong C, nếu hai biến cấu trúc thuộc cùng kiểu (cùng kiểu `struct`), bạn có thể gán giá trị của một biến cho biến kia bằng toán tử gán đơn giản (`=`).

- Ví dụ:

```
```c
struct Person {
    char name[20];
    int age;
};
struct Person p1 = {"Grok", 25};
struct Person p2;
p2 = p1; // Gán toàn bộ giá trị của p1 cho p2
```
```

- Sau phép gán, `p2.name` là "Grok" và `p2.age` là 25.

4. **\*\*It is impossible to have one structure within another structure. (TRUE / FALSE)\*\***

**\*\*Trả lời\*\*:** **\*\*FALSE\*\***.

**\*\*Giải thích\*\*:**

- Trong C, hoàn toàn có thể có một cấu trúc lồng (nested structure) bên trong một cấu trúc khác.

- Ví dụ:

```
```c
struct Date {
    int day, month, year;
};
struct Person {
    char name[20];
    struct Date dob; // Cấu trúc lồng
};
struct Person p;
p.dob.day = 22; // Truy cập phần tử của cấu trúc lồng
```
```

- Cấu trúc `Date` được lồng trong `Person`, nên điều này là khả thi.

5. **\*\*A new data type name can be defined by using the \_\_\_\_\_ keyword.\*\***

**\*\*Trả lời\*\*:** A new data type name can be defined by using the **\*\*typedef\*\*** keyword.

**\*\*Giải thích\*\*:**

- Từ khóa `typedef` trong C được dùng để tạo một tên kiểu dữ liệu mới (alias) cho một kiểu dữ liệu hiện có, bao gồm cả kiểu cấu trúc.

- Ví dụ:

```
```c
typedef struct {
    char name[20];
    int age;
} Person;
Person p; // Sử dụng Person thay vì struct
```
```

- `typedef` giúp mã dễ đọc hơn bằng cách tạo tên kiểu dữ liệu ngắn gọn.

6. **\*\*In bubble sort, the \_\_\_\_\_ elements are compared.\*\***

**\*\*Trả lời\*\*:** In bubble sort, the **\*\*adjacent\*\*** elements are compared.

**\*\*Giải thích\*\*:**

- Trong thuật toán sắp xếp nổi bọt (bubble sort), các phần tử **\*\*kề nhau\*\*** (adjacent elements) được so sánh và hoán đổi nếu chúng không đúng thứ tự. Quá trình này được lặp lại cho đến khi mảng được sắp xếp.

- Ví dụ: Với mảng `[5, 3, 8]`, bubble sort so sánh `5` và `3` (hoán đổi thành `[3, 5, 8]`), sau đó so sánh `5` và `8` (không đổi).

7. **\*\*In an unsorted array, if an element has to be put in a particular sorted location, values are swapped. (TRUE / FALSE)\*\***

**\*\*Trả lời\*\*:** **\*\*TRUE\*\***.

**\*\*Giải thích\*\*:**

- Trong một mảng chưa sắp xếp, nếu cần đặt một phần tử vào vị trí đã sắp xếp (như trong các thuật toán sắp xếp như bubble sort hoặc insertion sort), các giá trị thường được hoán đổi (swapped) để đưa phần tử vào đúng vị trí.

- Ví dụ: Trong insertion sort, nếu một phần tử cần được chèn vào vị trí đúng, các phần tử lớn hơn sẽ được dịch chuyển (thông qua hoán đổi) để tạo chỗ cho phần tử đó.

---

### Tóm tắt câu trả lời:

1. A **\*\*structure\*\*** data type groups together a number of data items, which need not be of the same type.

2. Individual structure elements are referenced through the use of the **\*\*dot operator\*\***.

3. Values of one structure variable can be assigned to another variable of the same type using a simple assignment statement: **\*\*TRUE\*\***.

4. It is impossible to have one structure within another structure: **\*\*FALSE\*\***.

5. A new data type name can be defined by using the **\*\*typedef\*\*** keyword.

6. In bubble sort, the **\*\*adjacent\*\*** elements are compared.

7. In an unsorted array, if an element has to be put in a particular sorted location, values are swapped: **\*\*TRUE\*\***.

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

Dưới đây là câu trả lời cho 9 câu hỏi của bạn, kèm theo giải thích chi tiết:

1. **\*\*The two types of streams are the \_\_\_\_\_ and \_\_\_\_\_ streams.\*\***

**\*\*Trả lời\*\*:** The two types of streams are the **\*\*text\*\*** and **\*\*binary\*\*** streams.

**\*\*Giải thích\*\*:**

- Trong C, có hai loại luồng chính:

- **\*\*Text streams\*\***: Xử lý dữ liệu dưới dạng ký tự (human-readable), thường dùng cho tệp văn bản (`.txt`). Có thể có chuyển đổi ký tự như `\n` thành `\r\n` trên Windows.

- **\*\*Binary streams\*\***: Xử lý dữ liệu dưới dạng byte thô, không qua chuyển đổi, thường dùng cho tệp nhị phân (`.bin`, `.jpg`).

- Ví dụ: Mở tệp ở chế độ văn bản (`"r"`) hoặc nhị phân (`"rb"`).

2. **\*\*Open files are closed when a program crashes. (TRUE/False)\*\***



**\*\*Trả lời\*\*:** **\*\*FALSE\*\***.

**\*\*Giải thích\*\*:**

- Khi chương trình gặp sự cố (crash), các tệp đang mở không được tự động đóng. Điều này có thể dẫn đến mất dữ liệu hoặc tệp bị hỏng nếu chương trình đang ghi dữ liệu.

- Để đảm bảo an toàn, cần gọi `fclose()` để đóng tệp hoặc hệ điều hành sẽ xử lý khi chương trình kết thúc bình thường, nhưng không đảm bảo trong trường hợp crash.

3. **\*\*The \_\_\_\_\_ function opens a stream for use and links a file with that stream.\*\***

**\*\*Trả lời\*\*:** The **\*\*fopen\*\*** function opens a stream for use and links a file with that stream.

**\*\*Giải thích\*\*:**

- Hàm `fopen()` trong C mở một tệp và tạo một luồng (stream) để đọc/ghi, liên kết tệp với luồng đó.

- Cú pháp: `FILE *fopen(const char *filename, const char *mode);`

- Ví dụ: `FILE *fp = fopen("data.txt", "w");` – mở tệp `data.txt` để ghi.

4. **\*\*The function used for writing characters to a file is \_\_\_\_\_.\*\***

**\*\*Trả lời\*\*:** The function used for writing characters to a file is **\*\*fputc\*\***.

**\*\*Giải thích\*\*:**

- Hàm `fputc()` được sử dụng để ghi từng ký tự vào tệp.

- Cú pháp: `int fputc(int c, FILE *stream);`

- Ví dụ:

```
```\nFILE *fp = fopen("data.txt", "w");\nfputc('A', fp); // Ghi ký tự 'A'\nfclose(fp);\n```
```

5. ****The fgetc() function considers a new line character as a part of the string. (TRUE/False)****

****Trả lời**:** ****TRUE****.

****Giải thích**:**

- Hàm `fgetc()` đọc một chuỗi từ tệp (hoặc stdin) và bao gồm ký tự dòng mới (`\\n`) nếu gặp, trước khi thêm ký tự kết thúc chuỗi (`\\0`).

- Ví dụ: Nếu tệp chứa dòng `"Hello\\n"`, `fgetc()` sẽ đọc `"Hello\\n"` vào bộ đệm.

```
```\nchar str[100];\nFILE *fp = fopen("data.txt", "r");\nfgetc(str, 100, fp); // str sẽ chứa "Hello\\n"\n```
```

6. **\*\*The \_\_\_\_\_ function resets the file position indicator to the beginning of the file.\*\***

**\*\*Trả lời\*\*:** The **\*\*rewind\*\*** function resets the file position indicator to the beginning of the file.

**\*\*Giải thích\*\*:**

- Hàm `rewind()` đặt con trỏ vị trí tệp (file position indicator) về đầu tệp.

- Cú pháp: `void rewind(FILE *stream);`

- Ví dụ:

```
```\n```
```

```
FILE *fp = fopen("data.txt", "r");
fseek(fp, 10, SEEK_SET); // Di chuyển con trỏ
rewind(fp); // Đặt lại về đầu tệp
...
```

7. ****Whenever a character is read from or written to the stream, the _____ is incremented.****

****Trả lời**:** Whenever a character is read from or written to the stream, the ****file position indicator**** is incremented.

****Giải thích**:**

- Con trỏ vị trí tệp (file position indicator) là một con trỏ nội bộ trong cấu trúc `FILE`, theo dõi vị trí hiện tại trong tệp. Mỗi khi đọc (`fgetc`) hoặc ghi (`fputc`) một ký tự, con trỏ này tăng lên tương ứng với số byte đã xử lý.

- Ví dụ: Sau khi đọc một ký tự bằng `fgetc()`, con trỏ di chuyển đến ký tự tiếp theo.

8. ****Files on which fread() and fwrite() operate must be opened in _____ mode.****

****Trả lời**:** Files on which `fread()` and `fwrite()` operate must be opened in ****binary**** mode.

****Giải thích**:**

- Hàm `fread()` và `fwrite()` được thiết kế để đọc/ghi dữ liệu nhị phân (raw bytes), nên tệp cần được mở ở chế độ nhị phân (binary mode) bằng cách thêm `"b"` vào chế độ mở (như `"rb"` để đọc, `"wb"` để ghi).

- Ví dụ:

```
```c
FILE *fp = fopen("data.bin", "wb"); // Mở ở chế độ nhị phân
int data = 123;
fwrite(&data, sizeof(int), 1, fp);
...
```

9. **\*\*The current location of the current active pointer can be found with the help of the \_\_\_\_\_ function.\*\***

**\*\*Trả lời\*\*:** The current location of the current active pointer can be found with the help of the **\*\*ftell\*\*** function.

**\*\*Giải thích\*\*:**

- Hàm `ftell()` trả về vị trí hiện tại của con trỏ vị trí tệp (file position indicator) dưới dạng số byte từ đầu tệp.

- Cú pháp: `long ftell(FILE *stream);`

- Ví dụ:

```
```c
FILE *fp = fopen("data.txt", "r");
fseek(fp, 5, SEEK_SET); // Di chuyển con trỏ
long pos = ftell(fp); // pos = 5
...
```

Tóm tắt câu trả lời:

1. The two types of streams are the ****text**** and ****binary**** streams.
2. Open files are closed when a program crashes: ****FALSE****.

3. The `**fopen**` function opens a stream for use and links a file with that stream.
4. The function used for writing characters to a file is `**fputc**`.
5. The `fgets()` function considers a new line character as a part of the string: `**TRUE**`.
6. The `**rewind**` function resets the file position indicator to the beginning of the file.
7. Whenever a character is read from or written to the stream, the `**file position indicator**` is incremented.
8. Files on which `fread()` and `fwrite()` operate must be opened in `**binary**` mode.
9. The current location of the current active pointer can be found with the help of the `**ftell**` function.

Hy vọng câu trả lời này rõ ràng và hữu ích! Nếu bạn cần giải thích thêm hoặc có câu hỏi khác, hãy cho tôi biết nhé!

```
#include <stdio.h>
#include <string.h>
```

// Hàm hiển thị menu

```
void displayMenu() {
    printf("*****\n");
    printf("*   Selecting appropriate action:           *\n");
    printf("*1. Question 1                               *\n");
    printf("*2. Question 2                               *\n");
    printf("*3. Quit program                             *\n");
    printf("*****\n");
    printf("Enter your choice: ");
}
```

// Hàm cho Question 1: Kiểm tra mảng có chứa toàn số lẻ không

```
void question1() {
    int n;
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid array size!\n");
        return;
    }

    int arr[n];
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

```
// Kiểm tra xem tất cả phần tử có phải số lẻ không
int allOdd = 1; // Giả sử tất cả là số lẻ
```

```

for (int i = 0; i < n; i++) {
    if (arr[i] % 2 == 0) { // Nếu gặp số chẵn
        allOdd = 0;
        break;
    }
}

if (allOdd) {
    printf("Array contains all odd numbers.\n");
} else {
    printf("Array does not contain all odd numbers.\n");
}
}

```

```

// Cấu trúc lưu thông tin sản phẩm
struct Product {
    int pro_no;
    char pro_name[50];
    float price;
};

```

```

// Hàm cho Question 2: Quản lý sản phẩm
void question2() {
    int n;
    printf("Enter the number of products in the factory: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid number of products!\n");
        return;
    }
}

```

```

struct Product products[n];
// Nhập thông tin sản phẩm
for (int i = 0; i < n; i++) {
    printf("Enter details for product %d:\n", i + 1);
    printf("Product number: ");
    scanf("%d", &products[i].pro_no);
    printf("Product name: ");
    scanf("%s", products[i].pro_name);
    printf("Price: ");
    scanf("%f", &products[i].price);
}

```

```

// Tìm kiếm sản phẩm theo mã
int pro_no;
int found = 0;
while (!found) {

```

```

printf("Enter product number to search: ");
scanf("%d", &pro_no);

// Tìm sản phẩm
for (int i = 0; i < n; i++) {
    if (products[i].pro_no == pro_no) {
        found = 1;
        printf("Product found!\n");
        printf("Product number: %d\n", products[i].pro_no);
        printf("Product name: %s\n", products[i].pro_name);
        printf("Price: %.2f\n", products[i].price);
        break;
    }
}

if (!found) {
    printf("Product not found! Please re-enter product number.\n");
}
}

int main() {
    int choice;
    do {
        displayMenu();
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                question1();
                break;
            case 2:
                question2();
                break;
            case 3:
                printf("Exiting program...\n");
                break;
            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 3);

    return 0;
}
#include <stdio.h>
#include <string.h>

// Hàm hiển thị menu

```

```

void displayMenu() {
    printf("*****\n");
    printf("*   Selecting appropriate action:           *\n");
    printf("*1. Question 1                                *\n");
    printf("*2. Question 2                                *\n");
    printf("*3. Quit program                             *\n");
    printf("*****\n");
    printf("Enter your choice: ");
}

```

// Hàm cho Question 1: Kiểm tra mảng có chứa toàn số lẻ không

```

void question1() {
    int n;
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid array size!\n");
        return;
    }

    int arr[n];
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Kiểm tra xem tất cả phần tử có phải số lẻ không
    int allOdd = 1; // Giả sử tất cả là số lẻ
    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 == 0) { // Nếu gặp số chẵn
            allOdd = 0;
            break;
        }
    }

    if (allOdd) {
        printf("Array contains all odd numbers.\n");
    } else {
        printf("Array does not contain all odd numbers.\n");
    }
}

```

// Cấu trúc lưu thông tin sản phẩm

```

struct Product {
    int pro_no;
    char pro_name[50];
    float price;
}

```

```
};
```

```
// Hàm cho Question 2: Quản lý sản phẩm
```

```
void question2() {
```

```
    int n;
```

```
    printf("Enter the number of products in the factory: ");
```

```
    scanf("%d", &n);
```

```
    if (n <= 0) {
```

```
        printf("Invalid number of products!\n");
```

```
        return;
```

```
    }
```

```
    struct Product products[n];
```

```
    // Nhập thông tin sản phẩm
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter details for product %d:\n", i + 1);
```

```
        printf("Product number: ");
```

```
        scanf("%d", &products[i].pro_no);
```

```
        printf("Product name: ");
```

```
        scanf("%s", products[i].pro_name);
```

```
        printf("Price: ");
```

```
        scanf("%f", &products[i].price);
```

```
    }
```

```
    // Tìm kiếm sản phẩm theo mã
```

```
    int pro_no;
```

```
    int found = 0;
```

```
    while (!found) {
```

```
        printf("Enter product number to search: ");
```

```
        scanf("%d", &pro_no);
```

```
        // Tìm sản phẩm
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (products[i].pro_no == pro_no) {
```

```
                found = 1;
```

```
                printf("Product found!\n");
```

```
                printf("Product number: %d\n", products[i].pro_no);
```

```
                printf("Product name: %s\n", products[i].pro_name);
```

```
                printf("Price: %.2f\n", products[i].price);
```

```
                break;
```

```
            }
```

```
        }
```

```
        if (!found) {
```

```
            printf("Product not found! Please re-enter product number.\n");
```

```
        }
```

```
    }
```

```
}

int main() {
    int choice;
    do {
        displayMenu();
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                question1();
                break;
            case 2:
                question2();
                break;
            case 3:
                printf("Exiting program...\n");
                break;
            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 3);

    return 0;
}
```